

Operations Manual

How to run, administer, and maintain the PEDMAS platform

www.pedmas.com · 2026-08-22

1. What you are operating

PEDMAS is an adaptive K–12 mathematics platform: 634 skills across Grades 1–12, 132 tap-through lessons, an adaptive placement test, daily practice with mastery tracking and spaced review, and a family subscription billed through Stripe.

Live site	<code>https://www.pedmas.com</code>
Local development	<code>http://localhost:3080</code> (npm run dev in C:\Users\buruf\Documents\Pedmas)
Code	<code>github.com/buruf/Pedmas</code> — every push to main deploys to production automatically via Vercel
Database	Neon Postgres (production) / JSON files in data\ (local development)
Payments	Stripe — currently in SANDBOX (test) mode
Email	Resend — key not yet configured; email features are dormant until it is

Push to main = deploy to production. There is no staging environment. Run npm test and npm run build locally before pushing.

2. Accounts and roles

- PARENT — created at signup by an adult. Owns child profiles, billing, and consent. Sees the parent dashboard and can delete any child profile or the whole account.
- STUDENT — a child profile inside a parent account. Children never have their own credentials or contact details.
- ADMIN — the platform owner (you). Sees everything at /admin plus all student profiles.

The admin login

The admin account is created automatically the first time the app runs with no admin present. Its credentials come from two environment variables:

PEDMAS_ADMIN_EMAIL	Admin email. Development fallback: <code>admin@pedmas.com</code>
PEDMAS_ADMIN_PASSWORD	Admin password. Development fallback: <code>pedmas-admin</code>

On your local machine the fallbacks work: log in at <http://localhost:3080/login> with `admin@pedmas.com` / `pedmas-admin`. In production the fallback has been verified NOT to work, which means your live admin uses the values stored in Vercel > your project > Settings > Environment Variables. If you have forgotten the password, look it up there.

Changing the environment variable later does NOT change an existing admin account — the account is only seeded once. To actually change the admin password: use the password-reset email flow (requires Resend), or edit the account row in Neon directly.

Two-factor authentication (do this before launch)

The admin account can read every family's data, so a stolen admin password is the worst thing that can happen to this service. Two-factor authentication makes that password useless on its own: after it is on, signing in asks for a six-digit code from your phone, and the password step no longer creates a session at all.

To turn it on: log in at `/admin`, find the Two-factor authentication panel at the top, press Turn on, then either paste the setup link into an authenticator app (Google Authenticator, Authy, 1Password) or type the key by hand. Enter the six-digit code it shows to prove the app works — nothing changes until that code verifies, so a mis-scanned setup costs you nothing.

You are then shown TEN RECOVERY CODES, once and never again. Save them somewhere you can reach WITHOUT your phone — a password manager, or printed and filed. Each works once, in place of the app. Without them, a lost phone means losing admin access entirely.

- Signing in: password, then the six-digit code. A recovery code can be typed in the same box if you do not have your phone; the app warns you how many remain.
- Turning it off, or issuing fresh recovery codes, both require a current code — so someone who steals a logged-in session cannot quietly remove your protection.
- It is admin-only by design. Parents recover their accounts by email; the admin account is the one worth protecting this way.

3. The admin console (`/admin`)

Log in as the admin, then open `/admin`. Non-admin accounts get “Admin only”. The console has five areas, top to bottom:

3.1 Overview cards

Accounts, Students, Placed students, Grades, Strands, Skills. Quick pulse of how much of the platform is in use.

3.2 Question generator preview

Pick any grade, skill and stage and see freshly generated questions exactly as students would, plus a health verdict (healthy / thin / very thin / fails validation) based on sampling: how many distinct questions the generator can produce and whether any fail validation. Use this when a parent reports a strange question — reproduce it here first.

3.3 Students table

Every student profile: grade, placement status, sessions completed, skills mastered, skills flagged struggling, and streak. Admin sees all students across all accounts.

3.4 Errors panel

- Every unhandled error — on the server or in a visitor's browser — is recorded automatically and grouped, with a count, the path, and when it last happened.
- “Fire a test error” throws a real server error on purpose. If it appears in the panel a moment later, monitoring is working end to end. Dismiss it afterwards.
- Dismiss removes a group once you have dealt with it. It reappears (count restarts) if the error happens again.
- When Resend is configured, a brand-new error also emails the admin address — at most one email every 6 hours.

3.5 Lesson effectiveness

For each lesson: first-try accuracy on that lesson's skills after children completed the lesson, versus a baseline (attempts before the lesson, plus children who never opened it), and the lift between them. Verdicts are withheld until each side has 25 attempts. Read it as a signal, not a controlled experiment — children choose whether to open lessons.

4. Everyday operations

Opening and closing registration

Registration is CLOSED by default and the homepage shows an under-construction banner. To open it: in Vercel add `REGISTRATION_OPEN=true` (exactly the word true) and redeploy. Remove it (or set anything else) to close again. Existing accounts always keep working either way.

Deploying

- Commit and push to main. Vercel builds and deploys automatically (about 2 minutes).
- Before pushing, always run locally: `npm test` (284 tests — includes every skill, every lesson step and the CAS audit) and `npm run build`.
- Never run `npm run build` while the local dev server is running — it corrupts the dev server's cache and buttons silently stop working. If that happens: stop the dev server, delete the `.next` folder, start it again.

Scheduled jobs

Two crons run automatically (configured in `vercel.json`): the weekly parent summary on Sundays at 16:00 UTC, and the dormant-account sweep daily at 03:30 UTC. Both refuse to run without the `CRON_SECRET` bearer token.

Weekly parent email

A Vercel cron calls `/api/cron/weekly-progress` every Sunday 16:00 UTC. It emails each parent a summary of every child's week, honours the account-level opt-out and the one-click unsubscribe link, and sends at most one email per account per day even if the cron retries. It does nothing until the Resend key is set.

Billing (Stripe)

- Pricing: \$11.99/month first child + \$5.99 each additional, up to 4 children, 7-day free trial.
- Currently in sandbox: use test card 4242 4242 4242 4242, any future expiry, any CVC.
- Parents manage cards, plan changes and cancellation themselves through the Stripe customer portal (Billing page in the app).
- Deleting an account automatically cancels its Stripe subscription immediately. If Stripe is unreachable at that moment, the deletion still happens and a loud log line tells you to cancel manually in the Stripe dashboard.
- Going live later requires: live Stripe keys + live webhook secret in Vercel, live price IDs, and the legal checklist below.

Data and deletion promises

- The privacy policy promises immediate, real deletion. The app delivers it: parents can delete a child profile or the whole account from the Account page; both are immediate hard deletes.
- The database is Neon Postgres: per-entity tables (accounts, students, auth_sessions, password_reset_tokens, rate_limits, stripe_events, error_events). The old `pedmas_rows` table is a frozen pre-migration backup — never write to it and never re-import it (that would resurrect deleted data).
- Region (US or international wording/units) is detected from the family's location on first visit and stamped on the account. There is no UI to change it yet; if a family asks, edit their account row's region field in Neon (US or INTL).

Dormant accounts are deleted automatically

Children's data may not be kept indefinitely, so a daily job erases accounts nobody uses. After 730 days (24 months) with no sign-in AND no child practice, the account holder is emailed a warning; 30 days later the account and every child profile on it are permanently deleted.

- Signing in resets the clock, and a child practising counts as activity just as much as a parent logging in.
- An account with a live subscription is NEVER deleted this way, whatever its billing state — a paying family is not dormant.
- Nothing is ever deleted without a warning first. Since warnings need email, NOTHING IS DELETED AT ALL until `RESEND_API_KEY` is set; the job says so plainly in its response.
- Deletion runs through exactly the same code as the parent's own delete button, so the two can never drift apart.

To see what it would do today without waiting for the cron, call it yourself: send GET to `/api/cron/retention` with header `Authorization: Bearer <CRON_SECRET>`. The reply reports how many accounts were examined, warned and purged.

5. Environment variables (Vercel)

DATABASE_URL	Neon Postgres connection string. Without it, production cannot store anything.
PEDMAS_ADMIN_EMAIL / _PASSWORD	Admin credentials, used once at first seeding.
REGISTRATION_OPEN	“true” opens signups. Anything else (or unset) keeps them closed.
AUTH_COOKIE_DOMAIN	.pedmas.com — makes login work across pedmas.com and www.pedmas.com. Do not remove.
NEXT_PUBLIC_APP_URL	https://www.pedmas.com — used in emails and Stripe redirect URLs.
STRIPE_SECRET_KEY	Stripe API key (currently sandbox <code>sk_test_...</code>).
STRIPE_WEBHOOK_SECRET	Signing secret for the Stripe webhook endpoint.
STRIPE_PRICE_FIRST_CHILD	Stripe price ID for the \$11.99 seat.
STRIPE_PRICE_ADDITIONAL_CHILD	Stripe price ID for the \$5.99 seat.
RESEND_API_KEY	Resend key. NOT YET SET — password reset, weekly summaries, placement report emails and error alerts are dormant until it is.
EMAIL_FROM	From address for outgoing mail, e.g. PEDMAS <hello@pedmas.com>.
CRON_SECRET	Bearer token protecting the weekly cron and the health check. Both refuse to run without it.

6. When something breaks

First steps

- The Errors panel at `/admin` — server and browser errors land there with counts and paths.
- Vercel dashboard > the project > Logs, for anything that crashed before the monitor could record it.
- The health check proves the store accepts writes: send GET <https://www.pedmas.com/api/health> with header `Authorization: Bearer <CRON_SECRET>`.

Rolling back a bad deploy

Two options: in the Vercel dashboard, promote the previous deployment (instant); or locally run `git revert <bad-commit>` and push, which redeploys the previous behaviour with history intact. Prefer revert — it keeps git and production telling the same story.

Known local-development gotchas

- `npm run build` while the dev server runs breaks the dev server silently (dead buttons, hydration gone). Stop server, delete `.next`, restart.
- Never bulk-edit source files with PowerShell text commands — they corrupt em dashes and curly quotes. Use an editor.
- Local data lives in `data\*.json`. Deleting those files resets your local accounts and students (production is untouched).

7. Before taking real money (legal checklist)

Tracked in detail in `docs/LEGAL-RATIONALE.md` in the repository. The blockers, in order:

- Fill the operator placeholders in `src/lib/legal.ts`: legal entity, postal address, monitored privacy email, governing jurisdiction. Counsel should advise on incorporating first.
- Accept the standard Data Processing Agreements with Stripe, Neon, Vercel and Resend (all self-serve).
- Decide launch geography (recommendation: US + Canada only) and state it in the terms.
- Have a lawyer review the Terms and Privacy Policy, then remove the draft banners.
- Write the short data-retention policy and information-security document the amended COPPA rule requires.

8. How question correctness is guaranteed

Three independent layers stand behind every question a child sees. Knowing what each covers tells you where a bug can still hide — and where to look first when a parent reports a wrong answer.

- Generation-time verification. Every question re-derives its own answer before it is served; anything failing that check is discarded and regenerated, never patched.
- Exhaustive sweeps on every test run. All 634 skills × 5 stages × 2 regions are generated and validated (about 19,000 questions), all 132 lessons are opened and every step tapped through, and every skill-to-lesson route is verified.
- The CAS second opinion, grades 8–12. An independent computer-algebra engine sharing no code with the generators re-derives answers from exactly the text a student sees. It closes the one gap the first layer cannot: a generator and its own checker agreeing on something wrong. It found precisely that on its first run — a derivative displayed as $23x^2$ where the answer is $6x^2$.

Running and reading the CAS audit

```
npm run audit:cas          # verdict: checked / passed / failed
npm run audit:cas:report   # per-family table with coverage
```

Expect FAILED 0. Roughly 4,500 of the ~8,000 senior questions per sweep are claimed by a checker; the rest are conceptual or vocabulary questions with nothing to recompute. A failure names the skill, stage, prompt and answer — decide whether it is a real content bug (fix the generator) or a reworded question (update the checker). Never loosen a checker to silence a failure: the test suite deliberately

feeds the auditor wrong answers it MUST catch, and an auditor that cannot fail proves nothing.

Run the audit after any change to grade 8–12 question generators. For the judgement no machine makes — does a question feel right for the grade — hand the Skill Coverage Checklist PDF to a mathematics teacher; its senior rows show each skill's CAS coverage, so the untagged ones are where human eyes are the only check.

9. Useful commands

```
cd C:\Users\buruf\Documents\Pedmas
npm run dev                # local server on http://localhost:3080
npm test                   # full test suite (284 tests)
npm run audit:cas          # independent CAS audit, grades 8-12 (expect FAILED 0)
npm run audit:cas:report   # per-family CAS coverage table
npm run build              # production build check (stop the dev server first)
npx tsx scripts/preview-weekly-email.ts # preview the weekly email locally
```

The three PDFs in docs/ regenerate from scripts/manual — run node manual.js, node testscript.js, or npx tsx checklist.ts in that folder after the product changes.

This manual describes the platform as of August 21, 2026 (commit 1647c20). When behaviour changes, regenerate or amend it — a manual that has drifted from the product is worse than none.